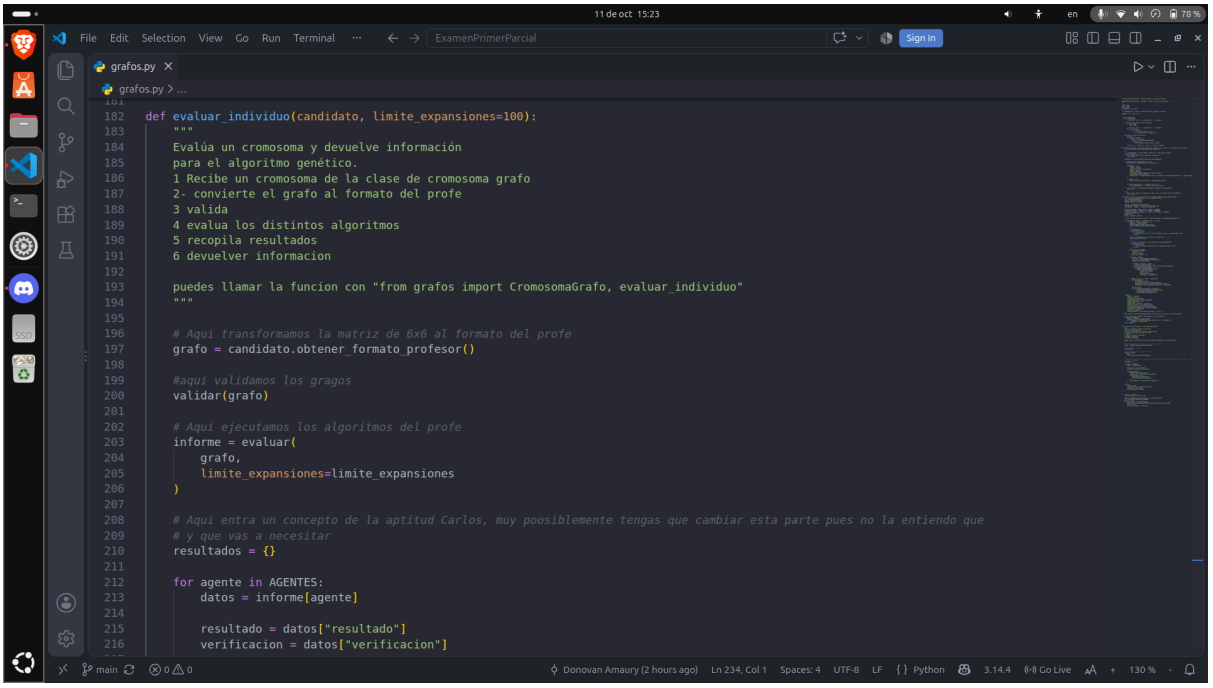
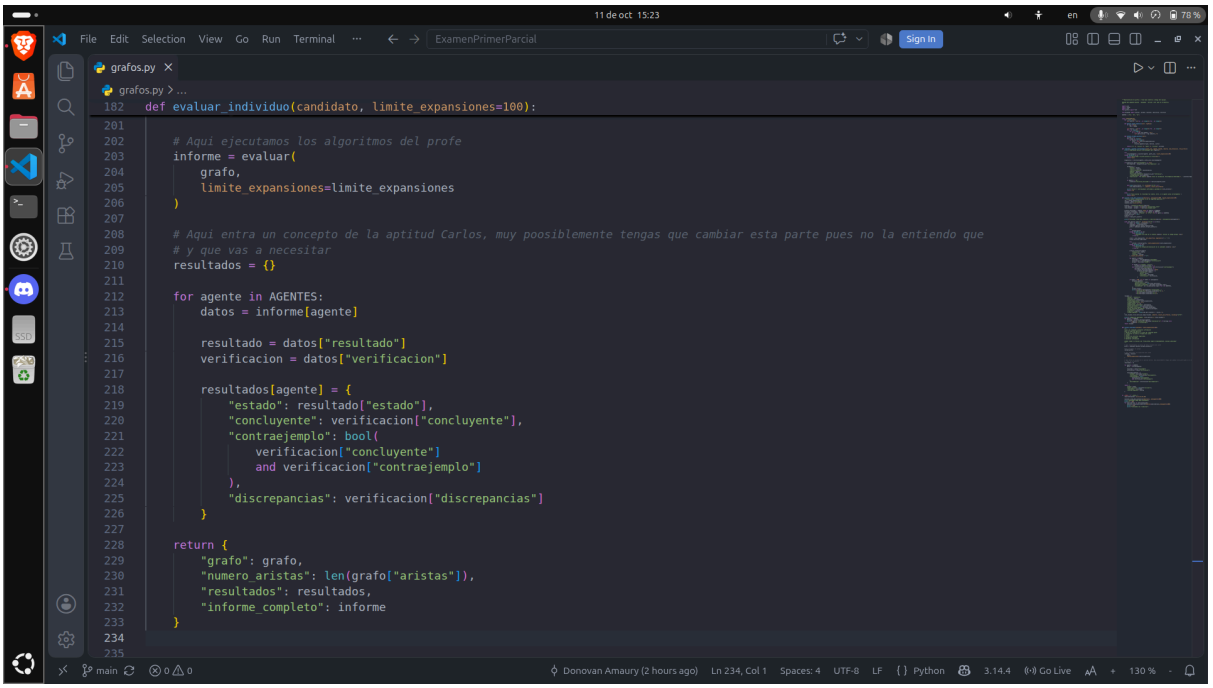
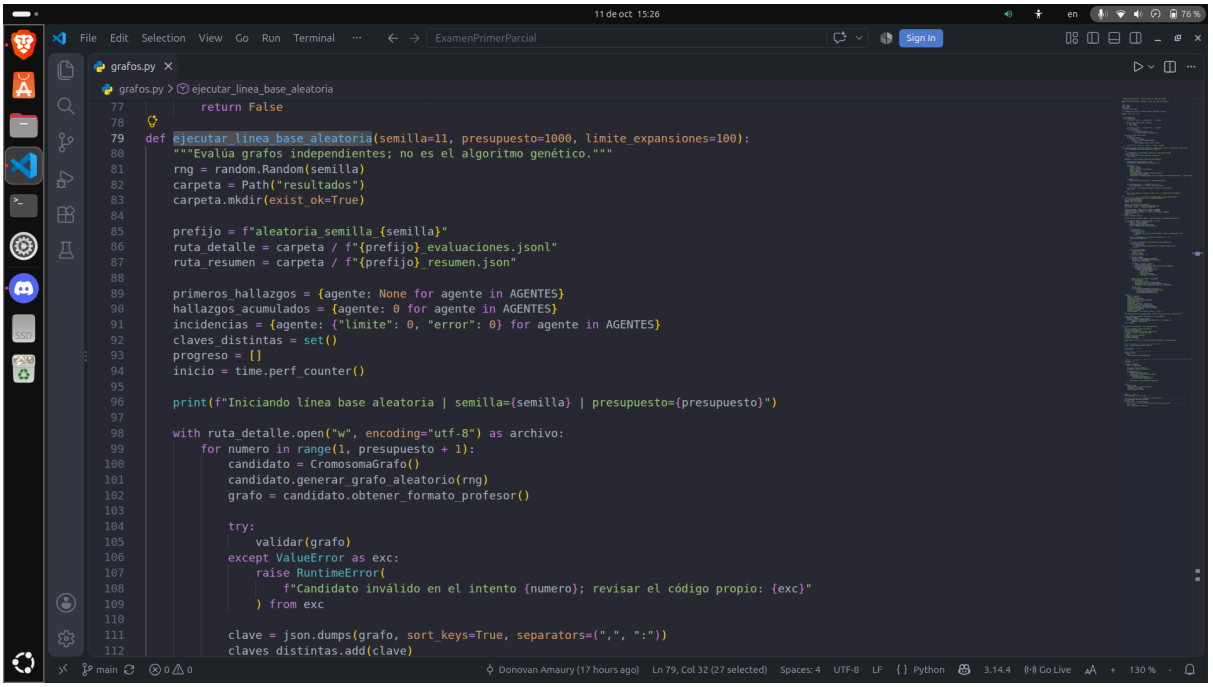
Contribuciones de cada integrante

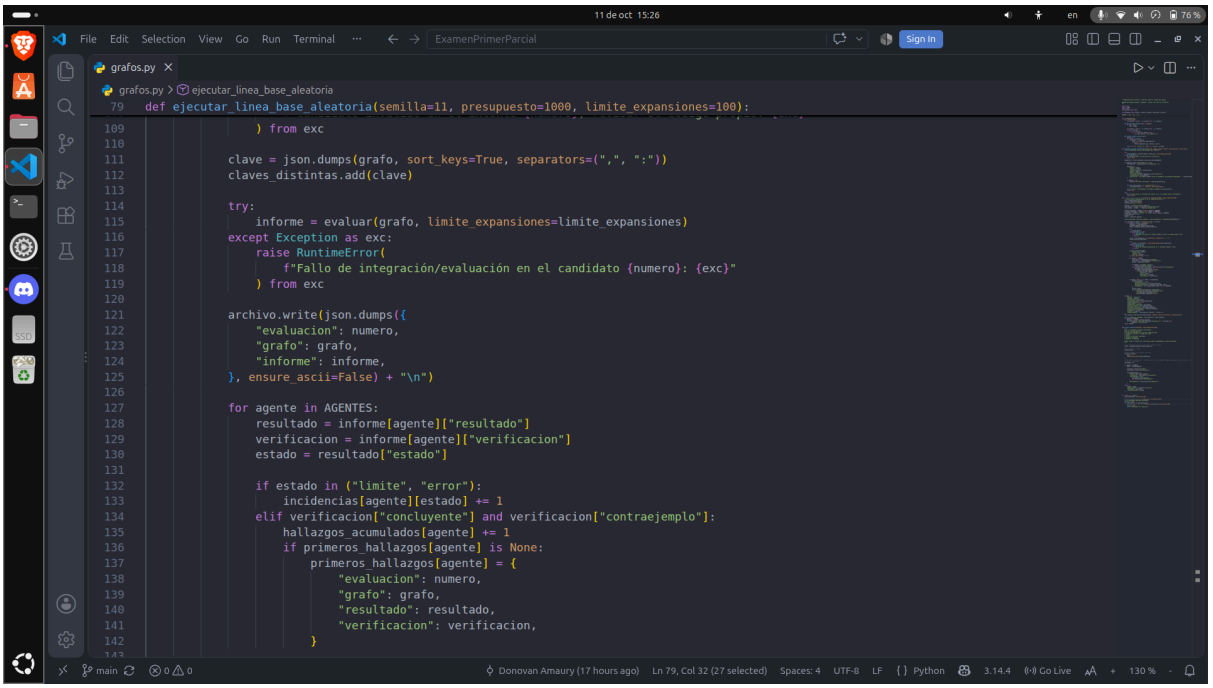
Donovan Amaury Alcantara Cruz

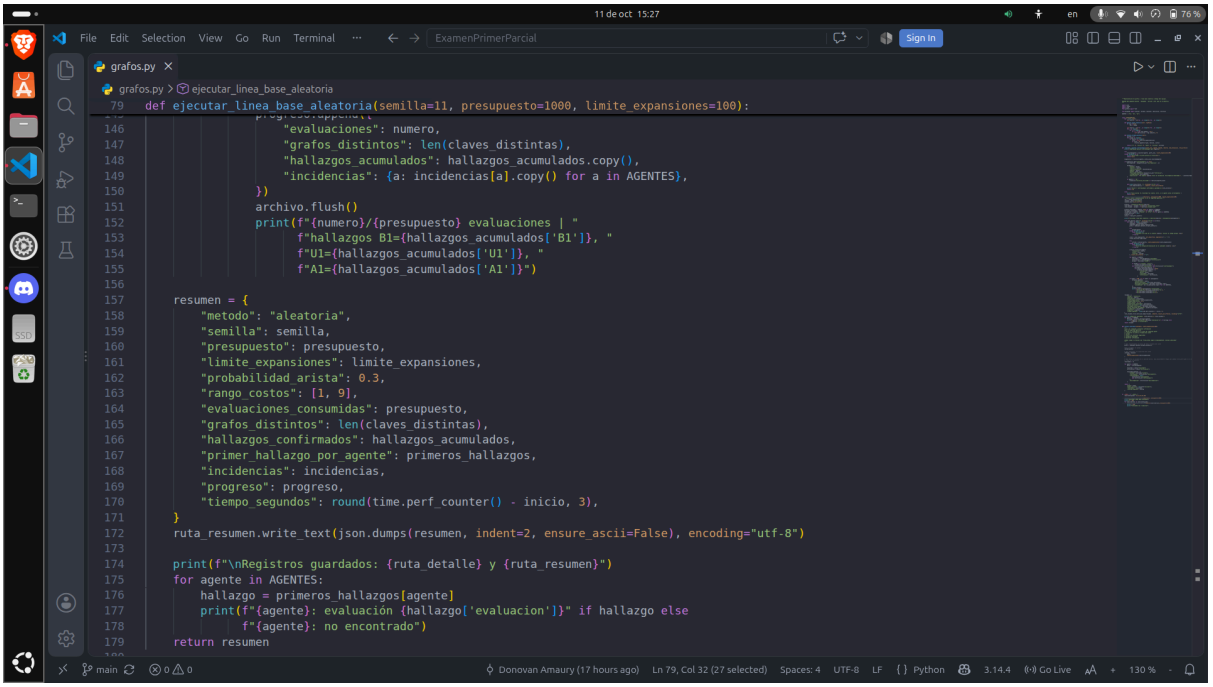
Aportó en la modificación de la clase CromosomasGrafo, contribuyendo en la funcion generar_grafo_aleatorio para que no se generarán los lazos y la función ejecutar_linea_base_aleatoria, hizo la función evaluar individuo(función puente) que genera y guarda la información para el algoritmo genético. También realizo el archivo README del repositorio. Realizó las pruebas que se exigen.





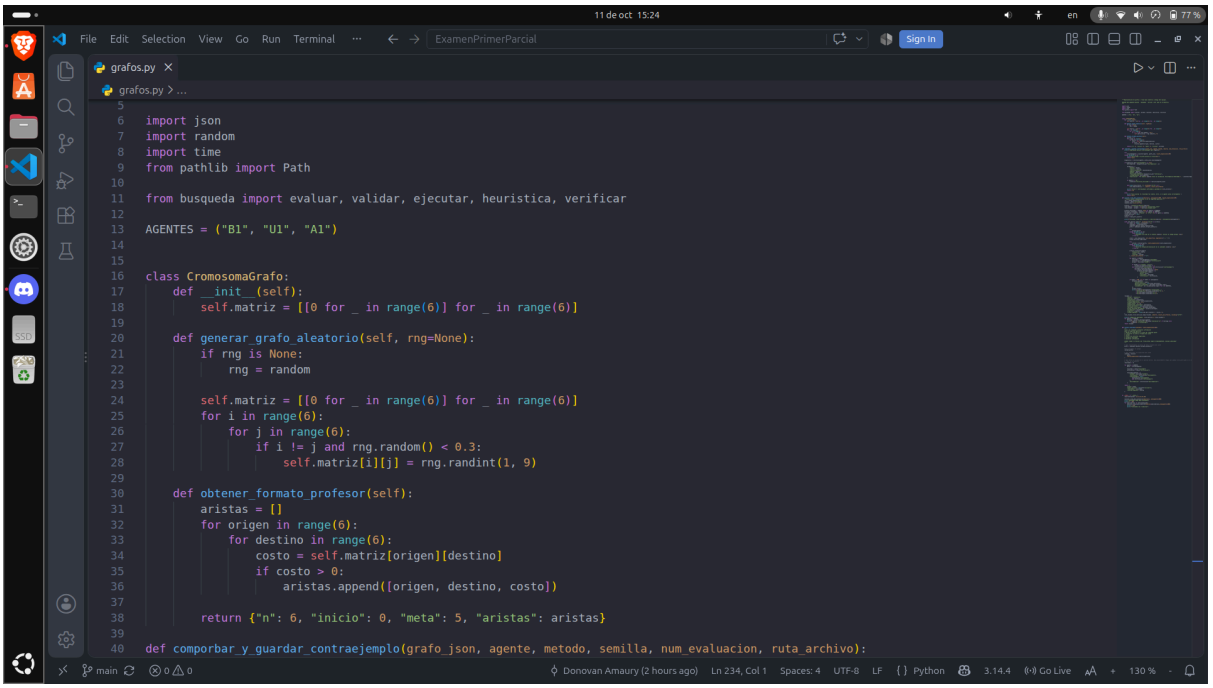

11 de oct 15:26
File Edit Selection View Go Run Terminal
ExamenPrimerParcial
Sign In
grafos.py
grafos.py > ejecutar_linea_base_aleatoria
77 return False
78
79 def ejecutar_linea_base_aleatoria(semilla=11, presupuesto=1000, limite_expansiones=100):
80 """Evalúa grafos independientes; no es el algoritmo genético."""
81 rng = random.Random(semilla)
82 carpeta = Path("resultados")
83 carpeta.mkdir(exist_ok=True)
84
85 prefijo = f"aleatoria_semilla_{semilla}"
86 ruta_detalle = carpeta / f"{prefijo}_evaluaciones.jsonl"
87 ruta_resumen = carpeta / f"{prefijo}_resumen.json"
88
89 primeros_hallazgos = {agente: None for agente in AGENTES}
90 hallazgos_acumulados = {agente: 0 for agente in AGENTES}
91 incidencias = {agente: {"limite": 0, "error": 0} for agente in AGENTES}
92 claves_distintas = set()
93 progreso = []
94 inicio = time.perf_counter()
95
96 print(f"Iniciando línea base aleatoria | semilla={semilla} | presupuesto={presupuesto}")
97
98 with ruta_detalle.open("w", encoding="utf-8") as archivo:
99 for numero in range(1, presupuesto + 1):
100 candidato = CromosomaGrafo()
101 candidato.generar_grafo_aleatorio(rng)
102 grafo = candidato.obtener_formato_profesor()
103
104 try:
105 validar(grafo)
106 except ValueError as exc:
107 raise RuntimeError(
108 f"Candidato inválido en el intento {numero}; revisar el código propio: {exc}"
109) from exc
110
111 clave = json.dumps(grafo, sort_keys=True, separators=(",", ":"))
112 claves_distintas.add(clave)
main
Donovan Amaury (17 hours ago)
Ln 79, Col 32 (27 selected)
Spaces: 4
UTF-8
LF
Python
3.14.4
Go Live
130 %

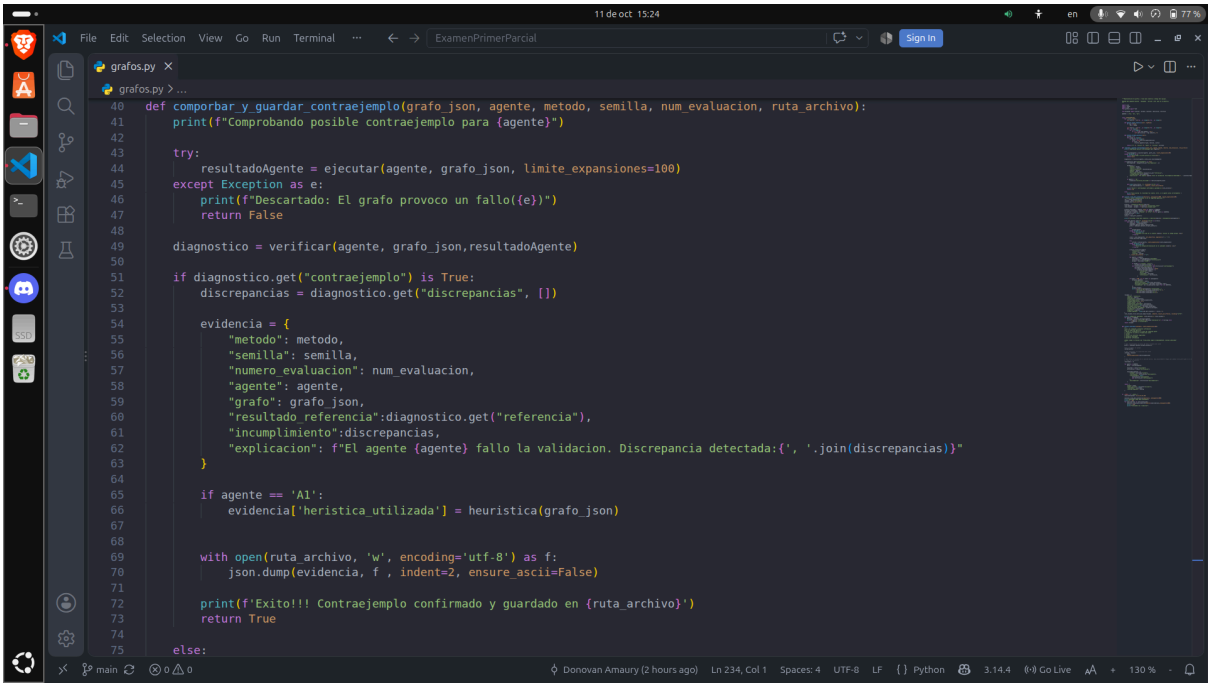

11 de oct 15:26
File Edit Selection View Go Run Terminal
ExamenPrimerParcial
Sign In
grafos.py
grafos.py > ejecutar_linea_base_aleatoria
79 def ejecutar_linea_base_aleatoria(semilla=11, presupuesto=1000, limite_expansiones=100):
109) from exc
110
111 clave = json.dumps(grafo, sort_keys=True, separators=(",", ":"))
112 claves_distintas.add(clave)
113
114 try:
115 informe = evaluar(grafo, limite_expansiones=limite_expansiones)
116 except Exception as exc:
117 raise RuntimeError(
118 f"Fallo de integración/evaluación en el candidato {numero}: {exc}"
119) from exc
120
121 archivo.write(json.dumps({
122 "evaluacion": numero,
123 "grafo": grafo,
124 "informe": informe,
125 }, ensure_ascii=False) + "\n")
126
127 for agente in AGENTES:
128 resultado = informe[agente]["resultado"]
129 verificacion = informe[agente]["verificacion"]
130 estado = resultado["estado"]
131
132 if estado in ("limite", "error"):
133 incidencias[agente][estado] += 1
134 elif verificacion["concluyente"] and verificacion["contraejemplo"]:
135 hallazgos_acumulados[agente] += 1
136 if primeros_hallazgos[agente] is None:
137 primeros_hallazgos[agente] = {
138 "evaluacion": numero,
139 "grafo": grafo,
140 "resultado": resultado,
141 "verificacion": verificacion,
142 }
main
Donovan Amaury (17 hours ago)
Ln 79, Col 32 (27 selected)
Spaces: 4
UTF-8
LF
Python
3.14.4
Go Live
130 %



Martin Gabriel Ramirez Hilario

Del archivo grafos.py, inicio la clase CromosomasGrafo, iniciando con las funciones generar_grafo_aleatorio, comprobar_guardar_contraejemplo y aporto parte del codigo de la funcion main para generar los archivos JSON para las semillas exigidas; tambien ayudo en la logica para la función obtener_formato_profesor.





```python
def comporbar_y_guardar_contraejemplo(grafo_json, agente, metodo, semilla, num_evaluacion, ruta_archivo):
    print(f"Comprobando posible contraejemplo para {agente}")

    try:
        resultadoAgente = ejecutar(agente, grafo_json, limite_expansiones=100)
    except Exception as e:
        print(f"Descartado: El grafo provoco un fallo({e})")
        return False

    diagnostico = verificar(agente, grafo_json,resultadoAgente)

    if diagnostico.get("contraejemplo") is True:
        discrepancias = diagnostico.get("discrepancias", [])

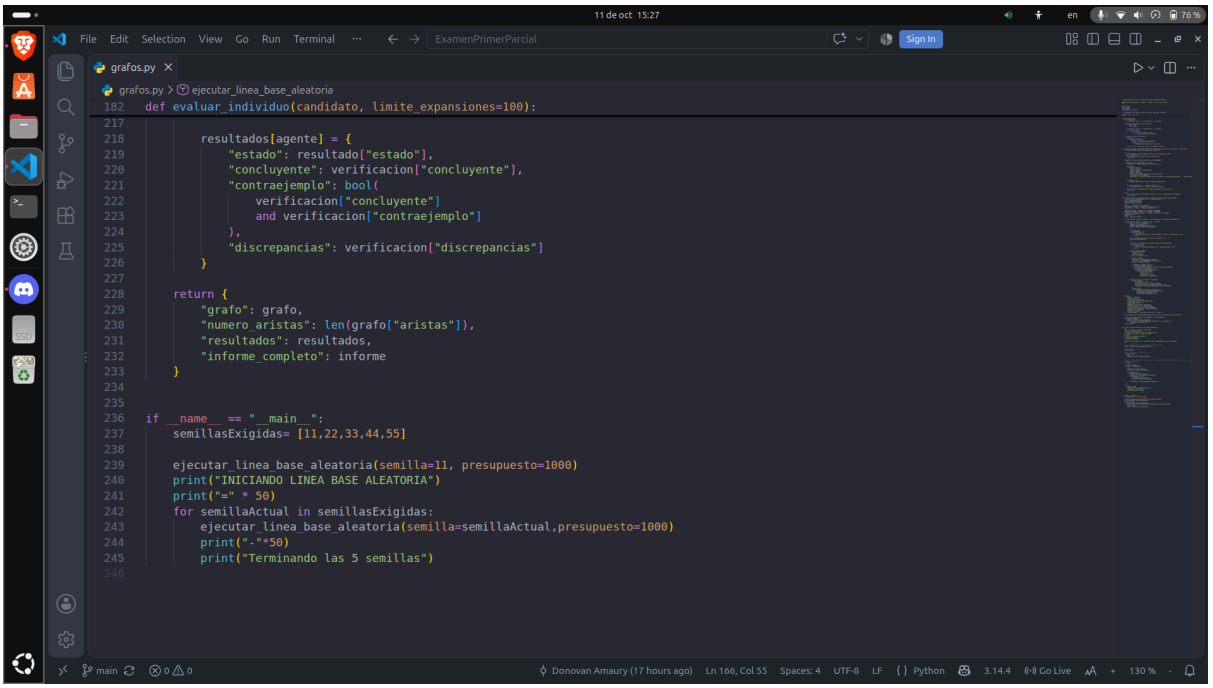
        evidencia = {
            "metodo": metodo,
            "semilla": semilla,
            "numero_evaluacion": num_evaluacion,
            "agente": agente,
            "grafo": grafo_json,
            "resultado referencia":diagnostico.get("referencia"),
            "incumplimiento":discrepancias,
            "explicacion": f"El agente {agente} fallo la validacion. Discrepancia detectada:{', '.join(discrepancias)}"
        }

        if agente == 'A1':
            evidencia['heristica_utilizada'] = heuristica(grafo_json)


        with open(ruta_archivo, 'w', encoding='utf-8') as f:
            json.dump(evidencia, f , indent=2, ensure_ascii=False)

        print(f'Exito!!! Contraejemplo confirmado y guardado en {ruta_archivo}')
        return True

    else:
```



```python
def evaluar_individuo(candidato, limite_expansiones=100):

            resultados[agente] = {
                "estado": resultado["estado"],
                "concluyente": verificacion["concluyente"],
                "contraejemplo": bool(
                    verificacion["concluyente"]
                    and verificacion["contraejemplo"]
                ),
                "discrepancias": verificacion["discrepancias"]
            }

    return {
        "grafo": grafo,
        "numero_aristas": len(grafo["aristas"]),
        "resultados": resultados,
        "informe_completo": informe
    }


if __name__ == "__main__":
    semillasExigidas= [11,22,33,44,55]

    ejecutar_linea_base_aleatoria(semilla=11, presupuesto=1000)
    print("INICIANDO LINEA BASE ALEATORIA")
    print("=" * 50)
    for semillaActual in semillasExigidas:
        ejecutar_linea_base_aleatoria(semilla=semillaActual,presupuesto=1000)
        print("-"*50)
        print("Terminando las 5 semillas")
```

Carlos Alberto de la Cruz Castelan
Creo el archivo GA,py en el cual se encuentran todas las funciones necesarias para realizar las operaciones de selección cruza y mutación de los individuos
Función de aptitud
Con esta función utiliza el informe generado de cada grafo para asignarle una aptitud con la cual decidiremos cual es el mejor candidato para la cruza

```python
def calculoAptitud (grafo, informe):
    aptitud = 0.0

    for agente in ["B!", "U1", "A1"]:
        if agente not in informe: continue

        reporte = informe[agente]
        estado = reporte["resultado"].get("estado")
        esContraejemplo = reporte["verificacion"].get("contraejemplo",False)

        if esContraejemplo == True:
            aptitud += 1000
        elif estado == "solucion":
            aptitud += 3
        elif estado == "sin_solucion":
            aptitud +=1
        elif estado == "limite":
            aptitud -=5

    cantAristas = len(grafo.get("aristas",[]))
    aptitud -= (cantAristas * 0.1)

    return aptitud
```

Función de cruza
Esta función cruza a los dos padres seleccionados para crear un nuevo grafo con las características mezcladas de sus padres

```python
def cruzar (grafoP1, grafoP2):
    hijo = {"n": 6, "inicio": 0, "meta": 5, "aristas": []}

    hijo["inicio"] = random.choice([grafoP1["inicio"],grafoP2["inicio"]])
    hijo["meta"] = random.choice([grafoP1["meta"],grafoP2["meta"]])

    aristasP1 = {(u,v):costo for u, v, costo in grafoP1["aristas"]}
    aristasP2 = {(u,v):costo for u, v, costo in grafoP2["aristas"]}


    for u in range(6):
        for v in range(6):
            if u==v:
                continue

            if random.random() < 0.5:
                costo = aristasP1.get((u, v), 0)

            else:
                costo = aristasP2.get((u, v), 0)

            if costo > 0:
                hijo["aristas"].appent([u, v, costo])

    return hijo
```

Función mutar
Esta función ayuda a que haya variedad entre los hijos y que el algoritmo no se vaya a estancar con los mismos genes

```python
def mutar(grafo, probMutacion = 0.15):

    mutado = copy.deepcopy(grafo)

    if random.random() < probMutacion:
        mutado["inicio"] = random.randint(0,5)

    if random.random() < probMutacion:
            mutado["meta"] = random.randint(0,5)

    if random.random() < probMutacion:

        accion = random.choice(["agregar", "eliminar", "cambCosto"])

        if accion == "agregar":

            u, v = random.sample(range(6), 2)

            costo = random.randint(0, 9)

            if not any(a[0] == u and a[1] == v for a in mutado["aristas"]):
                mutado["aristas"].append([u, v, costo])

        elif accion == "eliminar" and len(mutado["aristas"]) > 0:

            idx = random.randrange(len(mutado["aristas"]))

            mutado["aristas"].pop(idx)

        elif accion == "cambCosto" and len(mutado["arisras"]) > 0:

            idx = random.randrange(len(mutado["aristas"]))

            mutado["aristas"][idx][2] = random.randint(0,9)

    return mutado
```

Funcion de seleccion
Esta función selecciona a los primeros 3 lugares de toda la población para elegir al mejor de estos 3

```python
def selaccion(poblacion, k=3):

    participantes = random.sample(poblacion, k)

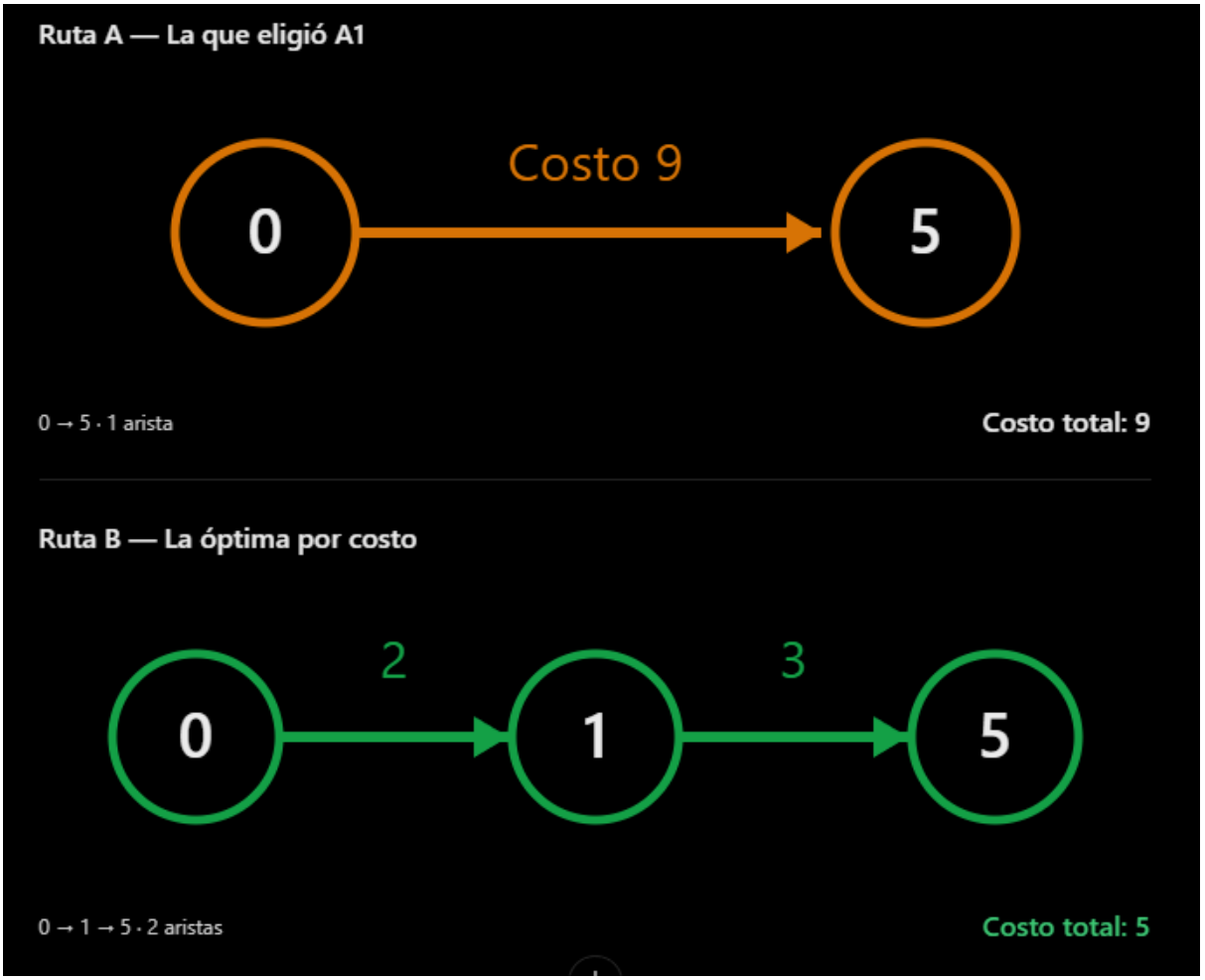
    ganador = max(participantes, key= lambda ind: ind["fitness"])

    return ganador
```

ContraEjemplos

Caso 1 - A1

- Ruta devuelta: 0 → 5
- Costo devuelto: 9
- Ruta alternativa: 0 → 1 → 5
- Costo alternativo: 5
- Heurística utilizada: [0, 0, 0, 0, 0, 0]
- Discrepancia: no_optimo
- Nombre del archivo: A1_semilla_11_eval_0043.json



Análisis del contraejemplo -A1

Durante la búsqueda aleatoria con semilla 11, en la evaluación numero 43 se identificó un grafo dirigido ponderado de seis nodos en el que A1, correspondiente al algoritmo A* devuelve una solución válida pero de costo superior al mínimo establecido
El agente devuelve la ruta del 0 al 5 con un costo de 9 cuando había una ruta mejor que es de 0 -> 1 -> 5 con costo de 5

El incumplimiento no se encuentra en si la ruta es válida, pues la conexion del 0 a 5 si existe, el error proviene de que una heurística de valor cero para todos los nodos:

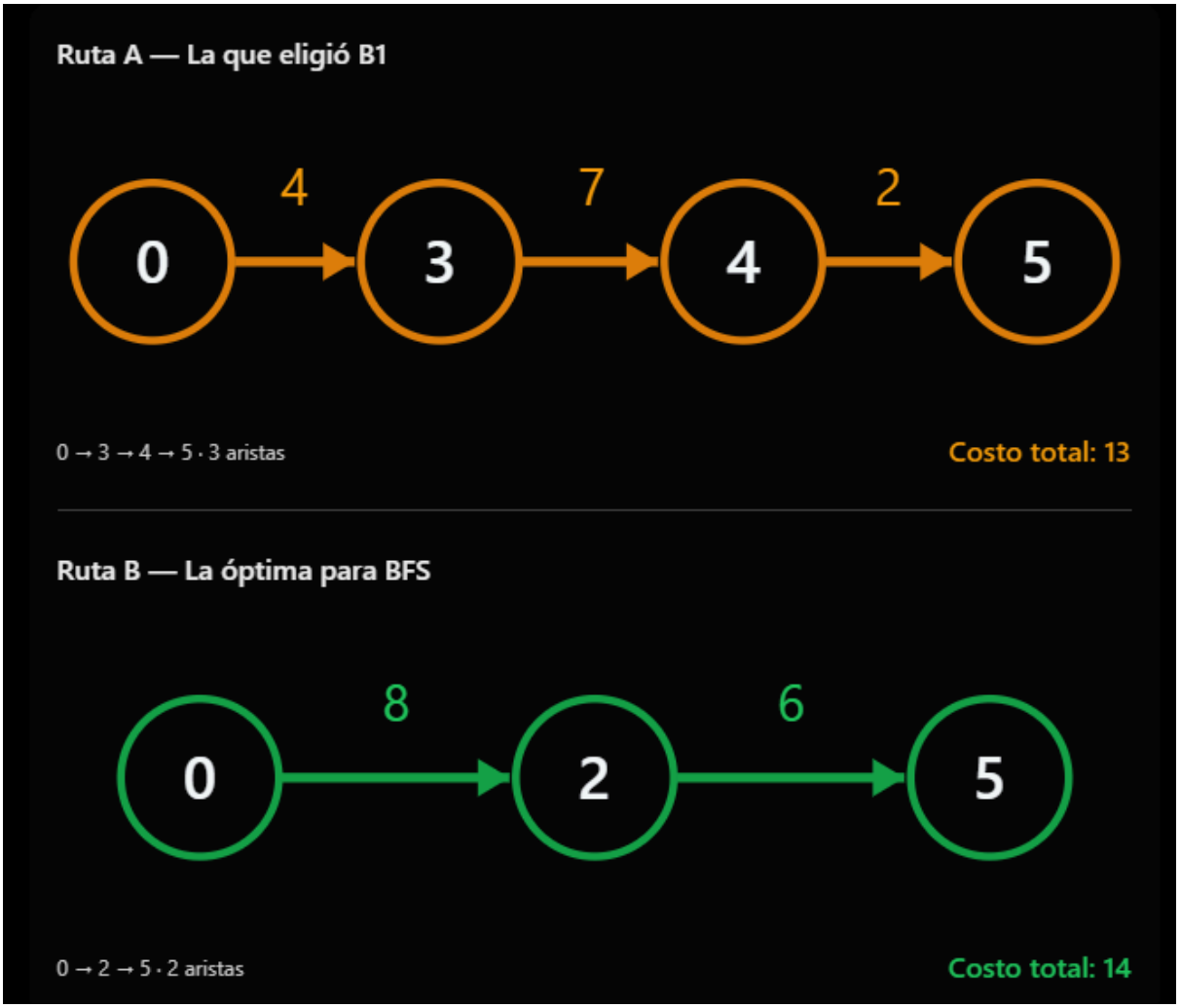
h = [0, 0, 0, 0, 0, 0]

El verificador del profesor identificó que no es óptimo debido al costo.

Este contraejemplo evidencia que la implementación A1 no respetó la garantía de optimalidad en el grafo valuado. Aunque encontró una ruta directa hacia la meta, omitió una alternativa de menor costo. El resultado permite demostrar un incumplimiento observable, sin que sea necesario identificar la instrucción interna que lo produjo.

Caso 2 - B1 (BFS)

- Semilla 11
- Evaluacion 188
- Ruta devuelta: 0 → 3 -> 4 -> 5
- Costo devuelto: 13
- Ruta alternativa: 0 → 2 → 5
- Costo alternativo: 14
- Discrepancia: no_optimo
- Nombre del archivo: B1_semilla_11_eval_0188.json



El algoritmo BFS explora los nodos por niveles de profundidad. Por ello debe regresar el menor número de pasos (aristas), debe encontrar una ruta con el menor número de aristas.

En nuestro grafo:

- La ruta devuelta tiene 3 aristas y un costo de 4 + 7 + 2 = 13.
- La ruta alternativa tiene 2 aristas y un costo de 8 + 6 = 14.

Aquí devuelve la ruta con costo menor cuando debía devolver la de menor aristas.

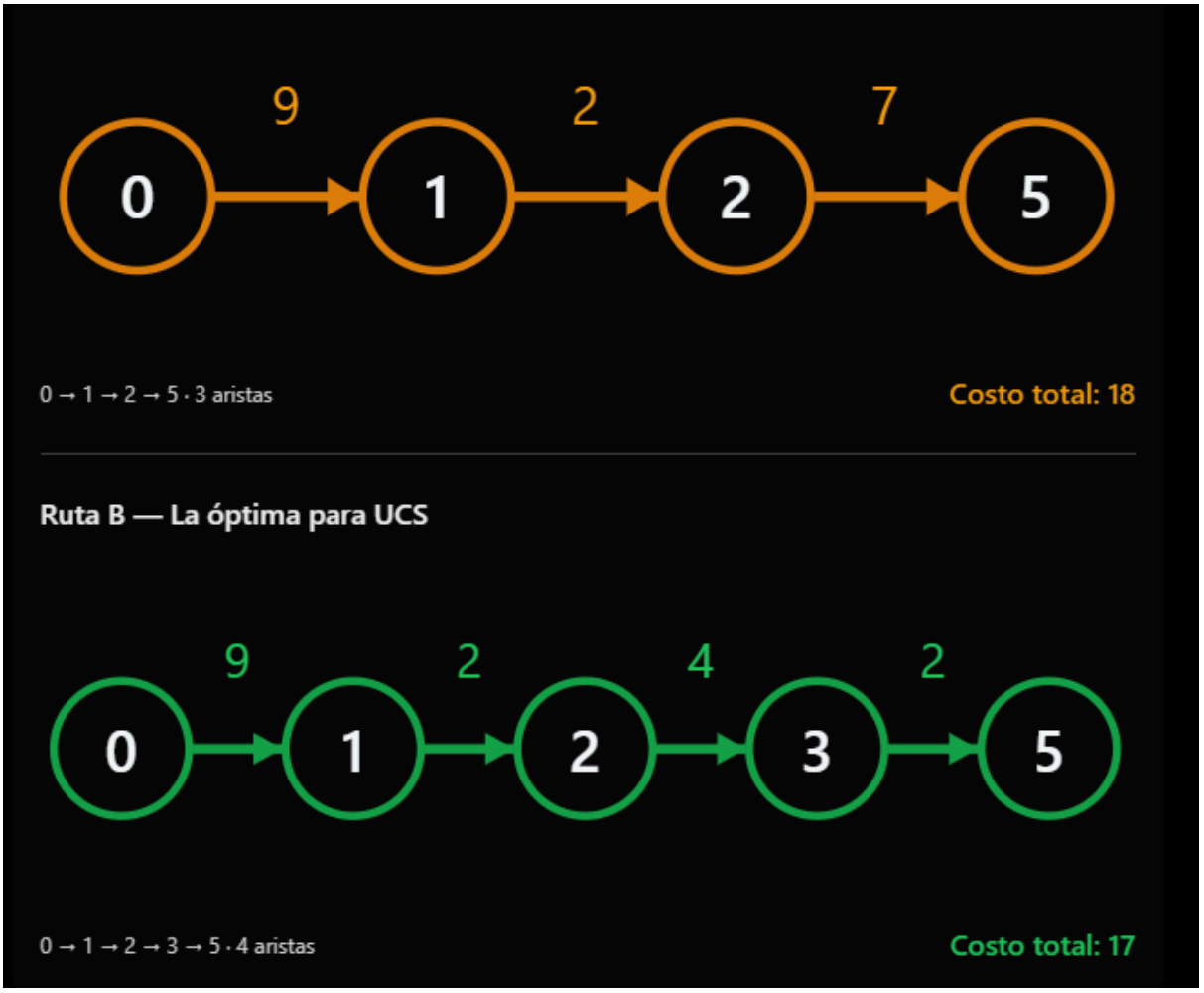
De hecho la referencia docente confirma que la distancia mínima es de dos aristas. El agente devuelve una ruta de tres, por lo que el verificador registra la discrepancia no óptima.

Podemos concluir que aunque devuelve una ruta existente y el costo declarado si es correcto no simple con dar el menor número de aristas.

Caso 3 - U1 (Uniform Cost Search)

- Semilla 11
- Evaluacion 2
- Ruta devuelta: 0 → 1 -> 2 -> 5

- Costo devuelto: 18
- Ruta alternativa: 0 → 1 → 2 -> 3 -> 5
- Costo alternativo: 17
- Discrepancia: no_optimo
- Nombre del archivo: U1_semilla_11_eval_0002.json



El UCS debe encontrar una ruta que minimice el costo acumulado, en este caso nos devolvió 18 cuando la mejor era la de 17
El verificador identifico que no era la ruta optima pues encontro la ruta con costo 17

Este contraejemplo demuestra que U1 incumplió su garantía de optimalidad. La ruta devuelta es válida y su costo declarado es correcto, pero existe una alternativa menos costosa.